Database Normalization

Third Normal Form (3NF)

Hospital Patient Visits — Assessment Problem

Course: Database Systems | Lab Manual — Task 8
Topic: 2NF → 3NF | XAMPP / MySQL 8.x

2NF ✓ → 3NF ✓

1. What is Third Normal Form (3NF)?

A table is in **Third Normal Form (3NF)** if and only if it is in 2NF **AND** no non-prime attribute is *transitively dependent* on the primary key. Every non-prime attribute must depend on '**the key, the whole key, and nothing but the key**.'

3NF Rule	The table must be in 2NF. For every non-trivial FD X → Y: either X is a superkey, or Y is a prime attribute. In simple terms: **no non-key column should determine another non-key column.**

2. BEFORE: 2NF Tables (Starting Point)

■■ The 2NF schema has 3 tables. The Doctor table still has a transitive dependency: DoctorID → DeptName → DeptHead. DeptHead depends on DeptName, not directly on DoctorID.

Table 1: Patient (No issues — stays unchanged)

PatientID (PK)	PatientName	PatientPhone
P-201	Hassan	0300-1112233
P-202	Mehreen	0301-4445566
P-203	Junaid	0302-7778899

Table 2: Doctor — ■■ Transitive Dependency Present

DoctorID (PK)	DoctorName	Specialty	DeptName	DeptHead ■■
D-30	Dr. Imran	Cardiology	**Heart Care**	**Dr. Tariq**
D-31	Dr. Asma	Dermatology	**Skin Clinic**	**Dr. Asma**

■ Red columns: DeptName → DeptHead is a transitive dependency. DeptHead does NOT depend on DoctorID — it depends on DeptName. This means DoctorID → DeptName → DeptHead (indirect/transitive chain).

Table 3: Visit (No issues — stays unchanged)

VisitID (PK)	VisitDate	PatientID (FK)	DoctorID (FK)	Diagnosis	Fee
V-9001	2026-04-10	P-201	D-30	Hypertension	2500
V-9002	2026-04-10	P-202	D-31	Eczema	2000
V-9003	2026-04-11	P-201	D-31	Allergy	2000
V-9004	2026-04-12	P-203	D-30	Arrhythmia	3000

3. Identifying the Transitive Dependency

DoctorID (PK) → **determines** → **DeptName (non-key)** → **determines** → **DeptHead (non-key)**

The problem: DeptHead does NOT directly depend on DoctorID (the PK). It depends on DeptName, which itself depends on DoctorID. This indirect chain (DoctorID → DeptName → DeptHead) is called a **transitive dependency**. 3NF forbids this — DeptHead must be moved to a table where it directly depends on the PK.

Why This Causes Problems:

Anomaly	Problem	Example
Update Anomaly	If Heart Care dept gets a new head, we update Doctor rows, not a dedicated dept row.	Heart Care's DeptHead appears in every D-30 row — update one place and others may not match.
Insertion Anomaly	Cannot record a new department until a doctor is assigned to it.	Cannot add Neurology department until a neurologist doctor is entered in Doctor table.
Deletion Anomaly	If the last doctor in a dept leaves, we lose department info.	Delete D-31 (Dr. Asma) → lose all info about 'Skin Clinic' department.

4. Decomposition Plan: 2NF → 3NF (4 Tables)

We split the Doctor table into two tables: **Doctor** (without dept head) and **Department** (with dept head). Patient and Visit tables remain unchanged.

#	Table Name	Columns	Change from 2NF
1	Patient	PatientID (PK), PatientName, PatientPhone	Unchanged from 2NF
2	Department	DeptName (PK), DeptHead	NEW — extracts transitive dep
3	Doctor	DoctorID (PK), DoctorName, Specialty, DeptName (FK)	Updated — DeptHead removed
4	Visit	VisitID (PK), VisitDate, PatientID (FK), DoctorID (FK), Diagnosis, Fee	Unchanged from 2NF

5. AFTER: Third Normal Form (3NF) — 4 Tables

■ All transitive dependencies eliminated | 4 tables | Every non-key attribute depends ONLY on the PK | No insertion / update / deletion anomalies

Table 1: Patient (Unchanged from 2NF)

PatientID (PK)	PatientName	PatientPhone
P-201	Hassan	0300-1112233
P-202	Mehreen	0301-4445566
P-203	Junaid	0302-7778899

Table 2: Department ■ NEW TABLE

DeptName is the PK. DeptHead directly depends on DeptName — no transitive chain.

DeptName (PK)	DeptHead
Heart Care	Dr. Tariq
Skin Clinic	Dr. Asma

Table 3: Doctor (DeptHead removed, DeptName is now FK)

DoctorID → DoctorName, Specialty, DeptName. DeptName is now a Foreign Key to Department.

DoctorID (PK)	DoctorName	Specialty	DeptName (FK)
D-30	Dr. Imran	Cardiology	Heart Care
D-31	Dr. Asma	Dermatology	Skin Clinic

Table 4: Visit (Unchanged from 2NF)

VisitID (PK)	VisitDate	PatientID (FK)	DoctorID (FK)	Diagnosis	Fee
V-9001	2026-04-10	P-201	D-30	Hypertension	2500
V-9002	2026-04-10	P-202	D-31	Eczema	2000
V-9003	2026-04-11	P-201	D-31	Allergy	2000
V-9004	2026-04-12	P-203	D-30	Arrhythmia	3000

6. Side-by-Side Comparison: 2NF vs 3NF

Feature	2NF (Before)	3NF (After)
No. of Tables	3 (Patient, Doctor, Visit)	4 (Patient, Department, Doctor, Visit)
Transitive Deps	■ DeptName → DeptHead in Doctor	■ Eliminated — Department table added
Doctor Table	DoctorID, Name, Specialty, DeptName, DeptHead	DoctorID, Name, Specialty, DeptName (FK)
DeptHead Storage	■ Repeated per doctor in Doctor	■ Stored once in Department table
Dept Insertion	■ Need a doctor to add dept info	■ Add directly to Department table

Doctor Update	■■ DeptHead change = update Doctor	■ Update Department table only
Deletion Safety	■ Delete last doctor → lose dept	■ Department record independent
All Anomalies	■■ Transitive dep anomalies remain	■ All anomalies eliminated
FK Relationships	Patient←Visit→Doctor	Patient←Visit→Doctor→Department

7. MySQL Implementation — 3NF

Step 1: Drop 2NF Doctor Table & Create 3NF Schema

```
-- ============================================================ -- DATABASE NORMALIZATION
— 3NF -- Hospital Patient Visits (Assessment Problem — Task 8) --
============================================================ USE hospital_normalization;
-- Must drop Visit first (FK to Doctor), then Doctor DROP TABLE IF EXISTS Visit; DROP
TABLE IF EXISTS Doctor; -- ■■ TABLE: Department (NEW — extracted from Doctor)
■■■■■■■■■■■■■■ CREATE TABLE Department ( DeptName VARCHAR(50) NOT NULL, -- PRIMARY KEY
DeptHead VARCHAR(50) NOT NULL, PRIMARY KEY (DeptName) ); -- ■■ TABLE: Doctor (DeptHead
removed, DeptName = FK) ■■■■■■■■■■■■■ CREATE TABLE Doctor ( DoctorID VARCHAR(10) NOT NULL,
DoctorName VARCHAR(50) NOT NULL, Specialty VARCHAR(50) NOT NULL, DeptName VARCHAR(50) NOT
NULL, -- FK → Department PRIMARY KEY (DoctorID), FOREIGN KEY (DeptName) REFERENCES
Department(DeptName) ON UPDATE CASCADE ON DELETE RESTRICT ); -- ■■ TABLE: Visit
(re-created with same FKs) ■■■■■■■■■■■■■■■■■■■■■■■■ CREATE TABLE Visit ( VisitID
VARCHAR(10) NOT NULL, VisitDate DATE NOT NULL, PatientID VARCHAR(10) NOT NULL, DoctorID
VARCHAR(10) NOT NULL, Diagnosis VARCHAR(100) NOT NULL, Fee INT NOT NULL, PRIMARY KEY
(VisitID), FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON UPDATE CASCADE ON
DELETE RESTRICT, FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID) ON UPDATE CASCADE ON
DELETE RESTRICT ); -- Patient table remains unchanged from 2NF
```

Step 2: Insert All Data (3NF Order)

```
-- Insert order matters: parent tables first, then children -- ■■ Patient (unchanged)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ INSERT INTO Patient VALUES ('P-201',
'Hassan', '0300-1112233'), ('P-202', 'Mehreen', '0301-4445566'), ('P-203', 'Junaid',
'0302-7778899'); -- ■■ Department (NEW table — insert before Doctor) ■■■■■■■■■■■■■■■■
INSERT INTO Department VALUES ('Heart Care', 'Dr. Tariq'), ('Skin Clinic', 'Dr. Asma'); --
■■ Doctor (DeptName references Department) ■■■■■■■■■■■■■■■■■■■■■■■■ INSERT INTO Doctor
VALUES ('D-30', 'Dr. Imran', 'Cardiology', 'Heart Care'), ('D-31', 'Dr. Asma',
'Dermatology', 'Skin Clinic'); -- ■■ Visit (references Patient and Doctor)
■■■■■■■■■■■■■■■■■■■■■■■■■■ INSERT INTO Visit VALUES ('V-9001', '2026-04-10', 'P-201',
'D-30', 'Hypertension', 2500), ('V-9002', '2026-04-10', 'P-202', 'D-31', 'Eczema', 2000),
('V-9003', '2026-04-11', 'P-201', 'D-31', 'Allergy', 2000), ('V-9004', '2026-04-12',
'P-203', 'D-30', 'Arrhythmia', 3000);
```

Step 3: Verification — Rebuild Original Report

```
-- ■■ Reconstruct original Table 8.1 from 4 joined tables ■■■■■■■ SELECT v.VisitID,
v.VisitDate, p.PatientID, p.PatientName, p.PatientPhone, d.DoctorID, d.DoctorName,
d.Specialty, d.DeptName, dept.DeptHead, -- now comes from Department table v.Diagnosis,
v.Fee FROM Visit v JOIN Patient p ON v.PatientID = p.PatientID JOIN Doctor d ON v.DoctorID
= d.DoctorID JOIN Department dept ON d.DeptName = dept.DeptName ORDER BY v.VisitDate; --
■■ Verify 3NF anomaly fixes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ -- 1. INSERT new
department WITHOUT any doctor (was impossible before) INSERT INTO Department VALUES
('Neurology', 'Dr. Khalid'); -- 2. UPDATE department head in ONE row only UPDATE
Department SET DeptHead = 'Dr. Saeed' WHERE DeptName = 'Heart Care'; -- 3. Show all tables
are clean SELECT * FROM Patient; SELECT * FROM Department; SELECT * FROM Doctor; SELECT *
FROM Visit;
```

8. Expected Output — Rebuilt Original Report

The JOIN query above reconstructs the full original dataset from 4 normalized tables:

VisitID	VisitDate	PatID	PatName	PatPhone	DocID	DocName	Specialty	DeptName	DeptHead	Diagnosis	Fee
V-9001	2026-04-10	P-201	Hassan	0300-1112233	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Hypertension	2500
V-9002	2026-04-10	P-202	Mehreen	0301-4445566	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Eczema	2000
V-9003	2026-04-11	P-201	Hassan	0300-1112233	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Allergy	2000
V-9004	2026-04-12	P-203	Junaid	0302-7778899	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Arrhythmia	3000

■ *All 4 original columns are reproduced. Data comes from 4 tables joined together — no redundancy in storage, full data on display.*

9. Final 3NF Compliance Checklist

■	**1NF Satisfied**	All cells atomic, no repeating groups, every row has a PK.
■	**2NF Satisfied**	No partial dependencies — each attribute depends on the full PK.
■	**3NF Satisfied**	No transitive dependencies — DeptHead moved to Department table.
■	**Insertion Anomaly Fixed**	Can add a new Department or Patient without a Visit.
■	**Update Anomaly Fixed**	Update DeptHead in Department once — no multi-row updates needed.
■	**Deletion Anomaly Fixed**	Delete all visits for a patient — Department/Doctor info preserved.
■	**Foreign Keys Enforced**	Visit→Patient, Visit→Doctor, Doctor→Department (referential integrity).
■	**Each Fact Stored Once**	Hassan's phone stored once. Dr. Imran's dept stored once.

10. Complete Normalization Summary

The Hospital Patient Visits dataset has been fully normalized from a single flat unnormalized table to a clean 3NF schema with **4 tables**. Every fact is now stored in exactly one place, all anomalies are eliminated, and referential integrity is enforced through foreign keys.

Normal Form	Structure	Key Change	Anomalies
UNF	1 table, 12 cols	No PK, repeating values	All anomalies present
1NF	1 table, 12 cols	PK = VisitID, atomic values	Atomicity fixed only
2NF	3 tables	Partial deps removed	Insertion/Update anomalies reduced
3NF	4 tables	Transitive deps removed	ALL anomalies eliminated ■

■ *Final schema: Patient ← Visit → Doctor → Department | 4 tables | 14 total columns across all tables | 0 redundant facts*